

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Основы программной инженерии
Лабораторная работа №4
Вариант 386127

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Проверил:
Кулинич Ярослав Вадимович,
Преподаватель практики

Санкт-Петербург 2026

Содержание

ЗАДАНИЕ.....	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ	4
Задание 1. Реализация mbean'ов	4
Задание 2. Мониторинг программы с помощью утилиты jconsole	9
Задание 3. Мониторинг и профилирование программы с помощью утилиты visualvm.....	11
Задание 4. Анализ, описание и устранение проблемы в программе httpunit.	14
ВЫВОД	28

Задание

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий процентное отношение "промахов" к общему числу кликов пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить время (в мс), прошедшее с момента запуска виртуальной машины.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Основные этапы выполнения

Задание 1. Реализация MBean'ов

- 1) MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.

▪ PointsStatsMBean.java

```
package org.wineeenottt.jmx;

import java.math.BigDecimal;

public interface PointsStatsMBean {

    int getTotalPoints();

    int getMissedPoints();

    int getOutOfBoundsPoints();

    void addPoints(boolean hit);

    void reset();

    void notifyIfOutOfBounds(BigDecimal x, BigDecimal y, BigDecimal r);

}
```

▪ PointsStats.java

```
package org.wineeenottt.jmx;

import jakarta.annotation.PreDestroy;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.context.Initialized;
import jakarta.enterprise.event.Observes;

import javax.management.MBeanServer;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;
import java.math.BigDecimal;
import java.math.RoundingMode;

@ApplicationScoped
public class PointsStats extends NotificationBroadcasterSupport
    implements PointsStatsMBean {

    private int totalPoints = 0;
    private int missedPoints = 0;
```

```

private int outOfBoundsPoints = 0;
int sqnNum = 0;

public void onStart(@Observes @Initialized(ApplicationScoped.class)
Object event) {
    try {
        MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
        ObjectName name = new
ObjectName("org.wineeenottt:type=PointsStats");
        if (!mbs.isRegistered(name)) {
            mbs.registerMBean(this, name);
        }
    } catch (Exception ignored) {}
}

@PreDestroy
public void unregister() {
    try {
        MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
        ObjectName name = new
ObjectName("org.wineeenottt:type=PointsStats");
        if (mbs.isRegistered(name)) {
            mbs.unregisterMBean(name);
        }
    } catch (Exception ignored) {}
}

@Override
public int getTotalPoints() {
    return totalPoints;
}

@Override
public int getMissedPoints() {
    return missedPoints;
}

@Override
public int getOutOfBoundsPoints() {
    return outOfBoundsPoints;
}

@Override
public void addPoints(boolean hit) {
    totalPoints++;
    if (!hit) missedPoints++;
}

@Override
public void reset() {
    totalPoints = 0;
    missedPoints = 0;
    outOfBoundsPoints = 0;
}

private BigDecimal calculateBound(BigDecimal r) {
    return r.multiply(new BigDecimal("300"))
        .divide(new BigDecimal("225"), 10, RoundingMode.HALF_UP);
}

private boolean isOutOfBounds(BigDecimal x, BigDecimal y, BigDecimal
bound) {
    BigDecimal min = bound.negate();

```

```

        return x.compareTo(min) < 0 || x.compareTo(bound) > 0
            || y.compareTo(min) < 0 || y.compareTo(bound) > 0;
    }

    private void sendOutOfBoundsNotification(BigDecimal x, BigDecimal y,
        BigDecimal r, BigDecimal bound) {

        outOfBoundsPoints++;

        BigDecimal min = bound.negate();
        String message = String.format(
            "Точка (x=%.2f, y=%.2f) вышла за пределы [%.2f..%.2f] x
[%.2f..%.2f], r=%.2f",
            x, y, min, bound, min, bound, r
        );

        Notification n = new Notification(
            "point.outOfDisplayBounds",
            this,
            sqnNum++,
            System.currentTimeMillis(),
            message
        );
        sendNotification(n);
    }

    @Override
    public void notifyIfOutOfBounds(BigDecimal x, BigDecimal y, BigDecimal r)
    {
        BigDecimal bound = calculateBound(r);
        if (isOutOfBounds(x, y, bound)) {
            sendOutOfBoundsNotification(x, y, r, bound);
        }
    }
}

```

- 2) MBean, определяющий процентное отношение "промахов" к общему числу кликов пользователя по координатной плоскости.

▪ MissRatioMBean.java

```

package org.wineeenottt.jmx;

public interface MissRatioMBean {

    void addClick(boolean hit);

    long getTotalClicks();

    long getMissCount();

    long getHitCount();

    double getMissRatioPercent();
}

```

▪ MissRatio.java

```
package org.wineeenottt.jmx;

import jakarta.annotation.PreDestroy;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.context.Initialized;
import jakarta.enterprise.event.Observes;

import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

@ApplicationScoped
public class MissRatio implements MissRatioMBean {

    private long totalClicks = 0;
    private long missCount = 0;

    public void onStart(@Observes @Initialized(ApplicationScoped.class)
Object event) {
        try {
            MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
            ObjectName name = new
ObjectName("org.wineeenottt:type=MissRatio");
            if (!mbs.isRegistered(name)) {
                mbs.registerMBean(this, name);
            }
        } catch (Exception ignored) {}
    }

    @PreDestroy
    public void unregister() {
        try {
            MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
            ObjectName name = new
ObjectName("org.wineeenottt:type=MissRatio");

            if (mbs.isRegistered(name)) {
                mbs.unregisterMBean(name);
            }
        } catch (Exception ignored) {}
    }

    @Override
    public void addClick(boolean hit) {
        totalClicks++;

        if (!hit) {
            missCount++;
        }
    }

    @Override
    public long getTotalClicks() {
        return totalClicks;
    }

    @Override
    public long getMissCount() {
        return missCount;
    }
}
```

```
@Override
public long getHitCount() {
    return totalClicks - missCount;
}

@Override
public double getMissRatioPercent() {
    if (totalClicks == 0) {
        return 0.0;
    }

    return (missCount * 100.0) / totalClicks;
}
}
```


Задание 2. Мониторинг программы с помощью утилиты JConsole

Мониторинг приложения, развёрнутого на сервере WildFly (процесс pid 4441), проводился с использованием утилиты **JConsole**. В ходе работы была задействована вкладка **MBeans**.

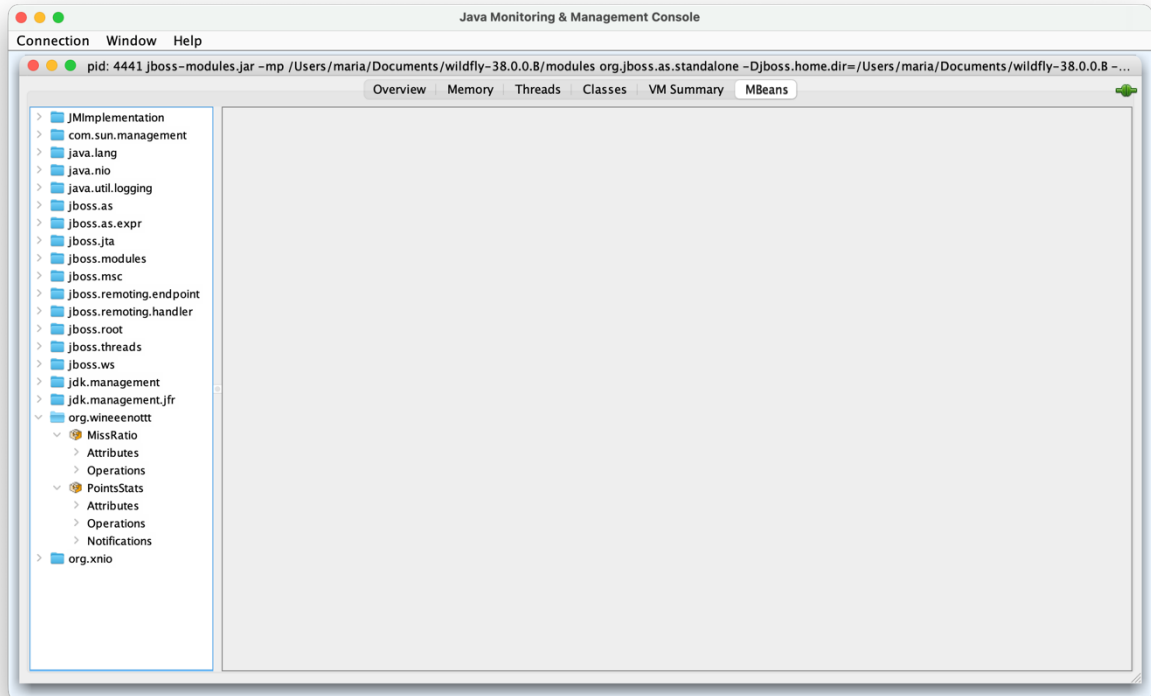


Рис. 1. Общий вид JConsole

1) Показания MBean-классов, разработанных в ходе выполнения задания 1

▪ **PointsStats**

Attribute values	
Name	Value
MissedPoints	57
OutOfBoundsPoints	11
TotalPoints	78

Рис. 2. Показания PointsStats

- **TotalPoints** — общее количество обработанных точек. Значение составило **78**, что полностью совпадает с TotalClicks из MBean MissRatio, подтверждая целостность данных между двумя классами.
- **MissedPoints** — счётчик промахов. Итоговое значение **57** совпадает с MissCount из MBean MissRatio.
- **OutOfBoundsPoints** — количество точек, вышедших за пределы отображаемой области. Значение составило **11**.

- **MissRatio**

Attribute values	
Name	Value
HitCount	21
MissCount	57
MissRatioPercent	73.07692307692308
TotalClicks	78

Рис. 3. Показания MissRatio

- **HitCount** — счётчик точек, попавших в допустимую область. Значение составило **21**.
- **MissCount** — счётчик промахов. Значение достигло **57**, что значительно превышает HitCount, указывая на то, что большинство сгенерированных точек не попадают в область.
- **MissRatioPercent** — процентное соотношение промахов к общему числу точек. Значение составило **73,077%**.
- **TotalClicks** — общее количество обработанных точек, равное **78**, что точно соответствует сумме HitCount и MissCount. Это подтверждает корректность и согласованность реализации MBean-класса.

Таким образом, разработанные MBean – классы MissRatio и PointsStats были успешно внедрены в веб-приложение. В ходе мониторинга было выявлено, что все атрибуты MBean-классов корректно функционируют. Значения атрибутов согласованы между двумя классами.

2) *Время (в мс), прошедшее с момента запуска виртуальной машины.*

Attribute value	
Name	Value
Uptime	775097

Рис. 4. Показания java.lang.Runtime

С помощью стандартного MBean java.lang.Runtime было определено время работы JVM — **775 097 мс** (примерно 12 минут 55 секунд). JConsole позволяет не только для мониторить пользовательские MBean'ы, но и получать системную информацию о состоянии и времени работы виртуальной машины Java.

Задание 3. Мониторинг и профилирование программы с помощью утилиты VisualVM

Мониторинг и профилирование приложения, развёрнутого на сервере WildFly (процесс pid 9795), проводились с использованием утилиты **VisualVM**. В ходе работы были задействованы вкладки **MBeans** и **Sampler**.

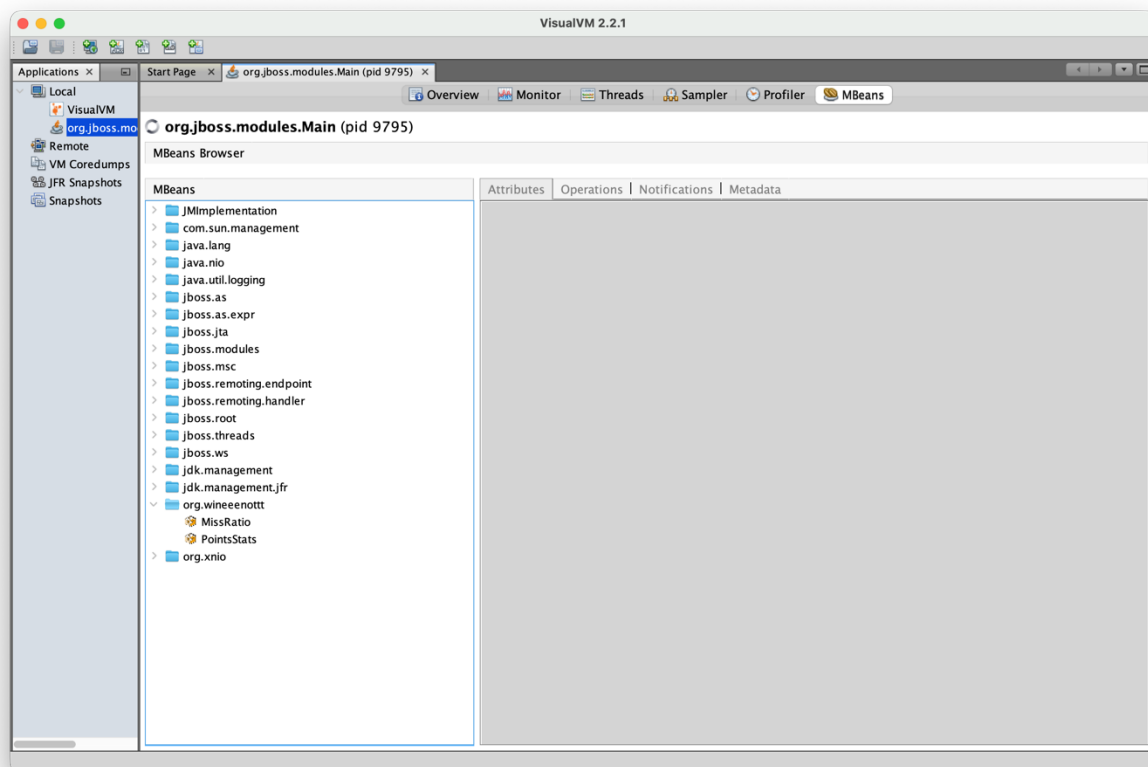


Рис. 5. Общий вид VisualVM

- 1) Графики изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени

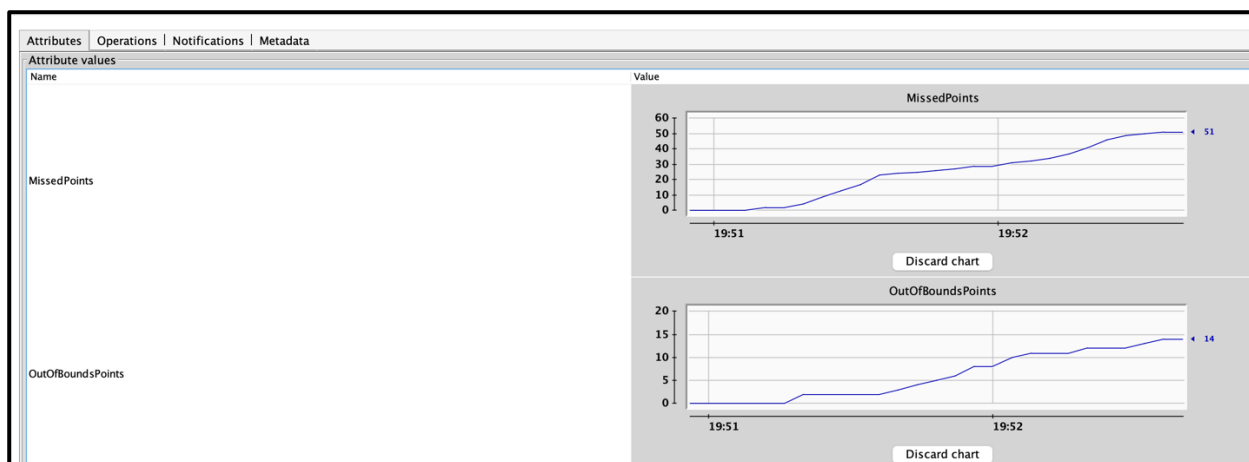


Рис. 6. Графики MBean PointsStats, часть 1 (атрибуты MissedPoints, OutOfBoundsPoints)

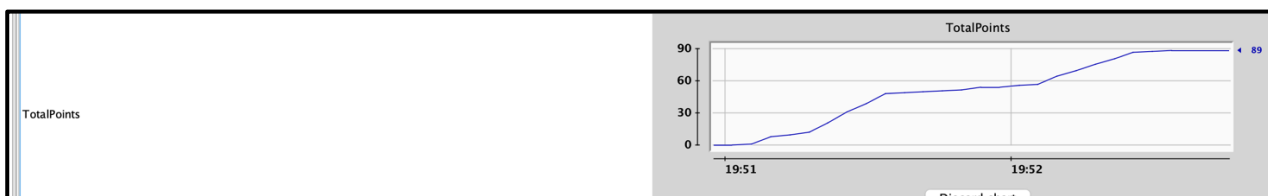


Рис. 7. График MBean PointsStats, часть 2 (атрибут TotalPoints)

- **TotalPoints** — общее количество обработанных точек. Значение непрерывно возрастало и к концу наблюдения составило **89**.
- **MissedPoints** — счётчик промахов. Итоговое значение **51** совпадает с MissCount из MBean MissRatio.
- **OutOfBoundsPoints** — количество точек, вышедших за пределы отображаемой области. Значение росло медленнее остальных счётчиков и достигло **14**.

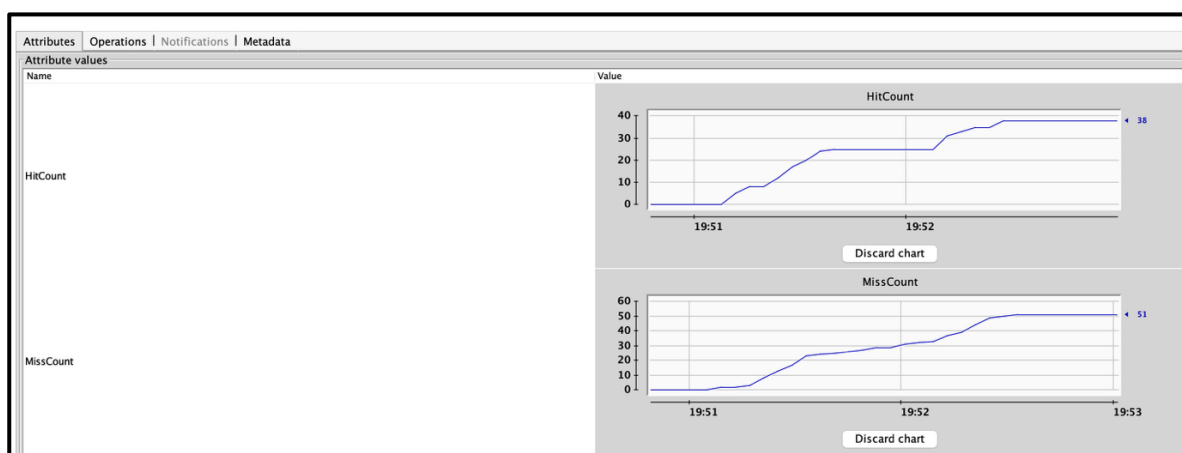


Рис. 8. Графики MBean MissRatio, часть 1 (атрибуты HitCount, MissCount)

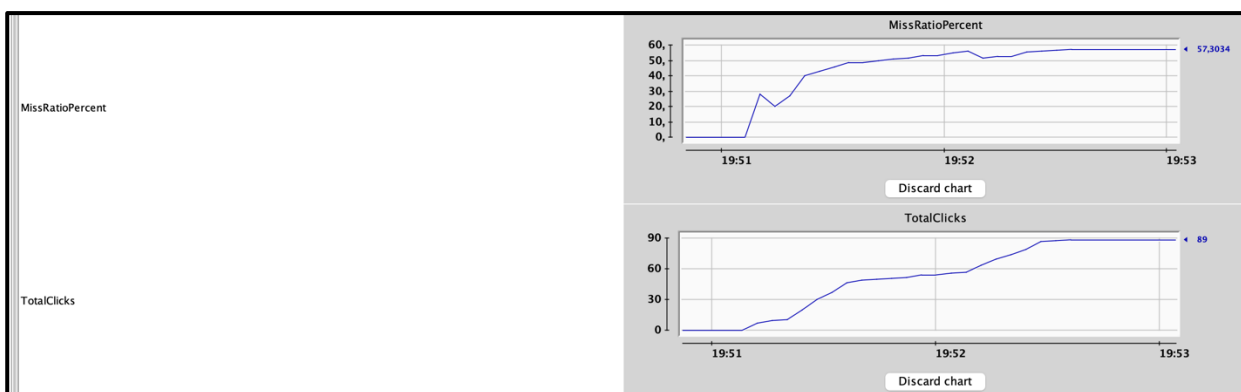


Рис. 9. Графики MBean MissRatio, часть 2 (атрибуты MissRatioPercent)

- **HitCount** — счётчик точек, попавших в область. За период наблюдения значение монотонно возросло от 0 до **38**.
- **MissCount** — счётчик промахов. За тот же период достиг значения **51**.

- **MissRatioPercent** — процентное соотношение промахов к общему числу точек. В начале наблюдения значение резко возросло от 0 до ~50%, после чего стабилизировалось на уровне **57,3%**.
- **TotalClicks** — общее количество обработанных точек. За период наблюдения значение возросло до **89**.

Таким образом, в ходе мониторинга было установлено, что все атрибуты MBean — классов функционируют корректно. Значения атрибутов согласованы между двумя классами.

2) Поток, потребляющего наибольший процент времени CPU.

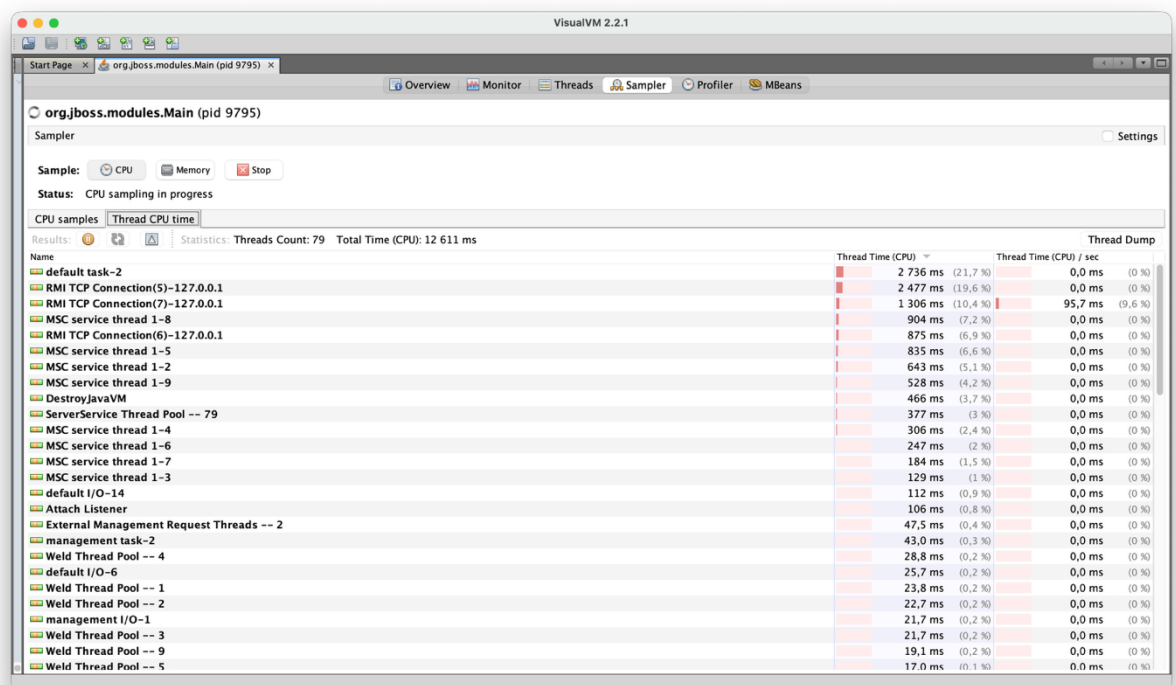


Рис. 10. Распределение времени процессора между потоками приложения

По результатам профилирования наибольший процент времени процессора потребляет поток **default-task-2** — 2 736 мс (21,7% от суммарного времени CPU).

Задание 4. Анализ, описание и устранение проблемы в программе HttpUnit.

Для начала профилирования приложение было запущено через IntelliJ IDEA.

Программа начала непрерывное создание объектов ServletUnitHttpResponse. В консоли наблюдался вывод строк вида Count: N...[_response =].

```
Count: 1[ _response = com.meterware.servletunit.ServletUnitHttpResponse@7085bdee]
Count: 2[ _response = com.meterware.servletunit.ServletUnitHttpResponse@6e06451e]
Count: 3[ _response = com.meterware.servletunit.ServletUnitHttpResponse@ff5b51f]
Count: 4[ _response = com.meterware.servletunit.ServletUnitHttpResponse@5649fd9b]
Count: 5[ _response = com.meterware.servletunit.ServletUnitHttpResponse@148080bb]
Count: 6[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3c0f93f1]
Count: 7[ _response = com.meterware.servletunit.ServletUnitHttpResponse@66480dd7]
Count: 8[ _response = com.meterware.servletunit.ServletUnitHttpResponse@20398b7c]
Count: 9[ _response = com.meterware.servletunit.ServletUnitHttpResponse@13a5fe33]
Count: 10[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3e9b1010]
```

Рис. 11. Вывод программы в терминале IntelliJ IDEA

```
Count: 5095[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3d1637ff]
Count: 5096[ _response = com.meterware.servletunit.ServletUnitHttpResponse@f0ead61]
Count: 5097[ _response = com.meterware.servletunit.ServletUnitHttpResponse@4d630ffa]
Count: 5098[ _response = com.meterware.servletunit.ServletUnitHttpResponse@175957f9]
Count: 5099[ _response = com.meterware.servletunit.ServletUnitHttpResponse@4fb5adb8]
Count: 5100[ _response = com.meterware.servletunit.ServletUnitHttpResponse@6cb890ac]
Count: 5101[ _response = com.meterware.servletunit.ServletUnitHttpResponse@1199ac5]
Count: 5102[ _response = com.meterware.servletunit.ServletUnitHttpResponse@40052266]
Count: 5103[ _response = com.meterware.servletunit.ServletUnitHttpResponse@1d6daa8a]
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.base/java.lang.invoke.LambdaForm.compileToBytecode(LambdaForm.java:841)
    at java.base/java.lang.invoke.LambdaForm.prepare(LambdaForm.java:799)
    at java.base/java.lang.invoke.MethodHandle.<init>(MethodHandle.java:484)
    at java.base/java.lang.invoke.BoundMethodHandle.<init>(BoundMethodHandle.java:52)
    at java.base/java.lang.invoke.BoundMethodHandle$Species_LLLLL.<init>(BoundMethodHandle$Species_LLLLL)
    at java.base/java.lang.invoke.BoundMethodHandle$Species_LLLLL.make(BoundMethodHandle$Species_LLLLL)
    at java.base/java.lang.invoke.LambdaForm$DMH/0x000003e00103ac00.invokeStatic(LambdaForm$DMH)
    at java.base/java.lang.invoke.BoundMethodHandle$Species_LLLL.copyWithExtendL(BoundMethodHandle$Species_LLLL)
    > <24 folded frames>
Process finished with exit code 1
```

Рис. 12. Завершение программы с ошибкой в терминале IntelliJ IDEA

Спустя некоторое время программа завершилась с ошибкой Java OutOfMemoryError: Java heap space, то есть нехватка памяти. Стек вызовов указывает на то, что ошибка произошла в момент компиляции лямбда-выражения (LambdaForm.compileToBytecode), то есть JVM не удалось выделить память, так как heap был уже полностью исчерпан. Это свидетельствует об утечке памяти.

- VisualVM

Запустим программу снова, и подключим к ее процессу утилиту VisualVM. Программа работала 17 минут 43 секунды и вновь завершилась с ошибкой переполнения памяти.

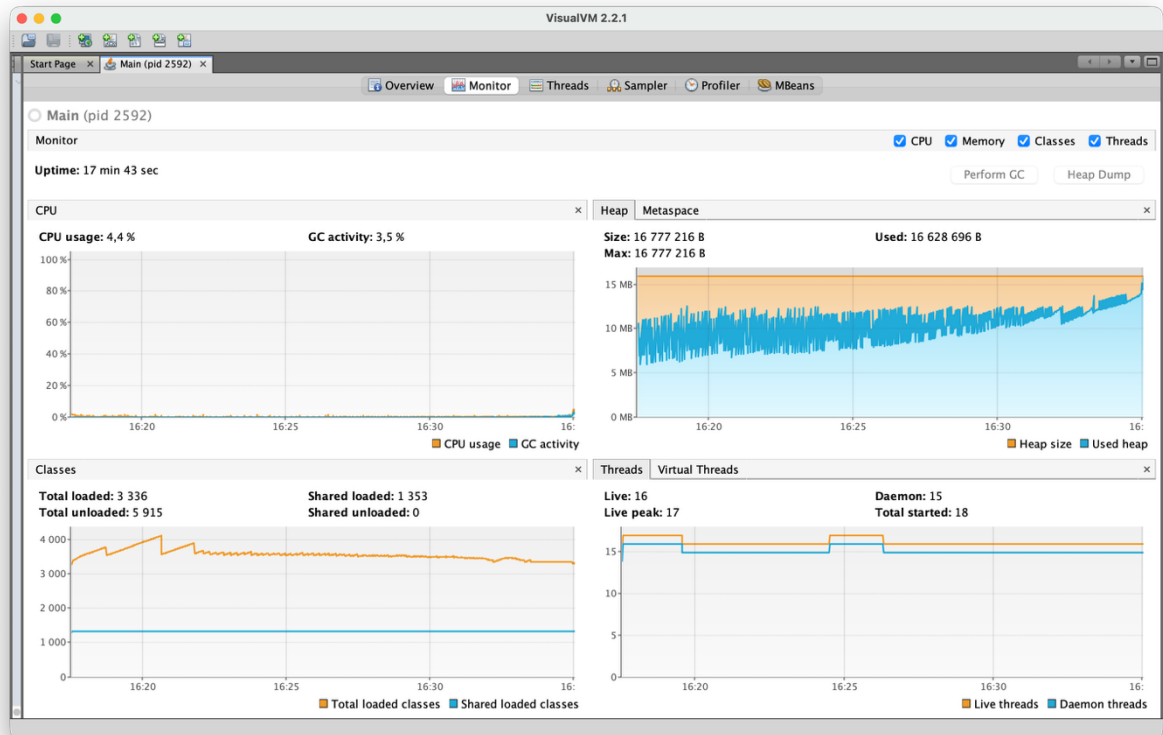


Рис. 13. Показатели CPU, heap, классов и потоков во время выполнения программы

Размер использованной памяти на момент завершения программы составил ~15,9 Мб, следовательно heap был заполнен. На протяжении всей работы программы используемая память росла. Пилообразный график говорит о постоянной работе сборщика мусора, однако каждый раз после очистки уровень потребления памяти оставался выше предыдущего. Это подтверждает утечку памяти.

Для локализации этой проблемы необходимо узнать, какие объекты занимают большую часть памяти. Для этого подробнее рассмотрим содержимое heap.

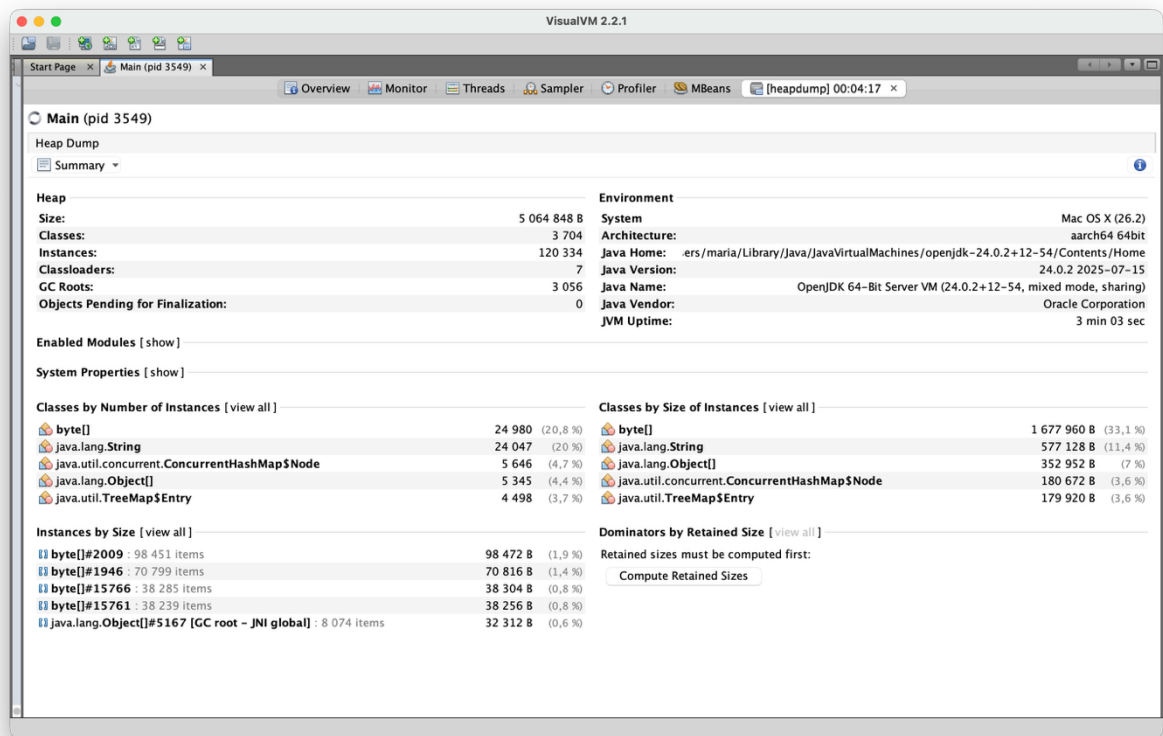


Рис. 14. Результаты Heap Dump

Проанализировав содержимое кучи, можно заметить, что наибольшую долю памяти занимают `byte[]` и объекты `java.lang.String`. При работе приложения на каждый запрос создаются экземпляры строк, они остаются не очищенными в памяти.

Рассмотрим подробнее объекты `java.lang.String`. При раскрытии данного класса в режиме "All Objects" становятся видны конкретные значения хранящихся строк.

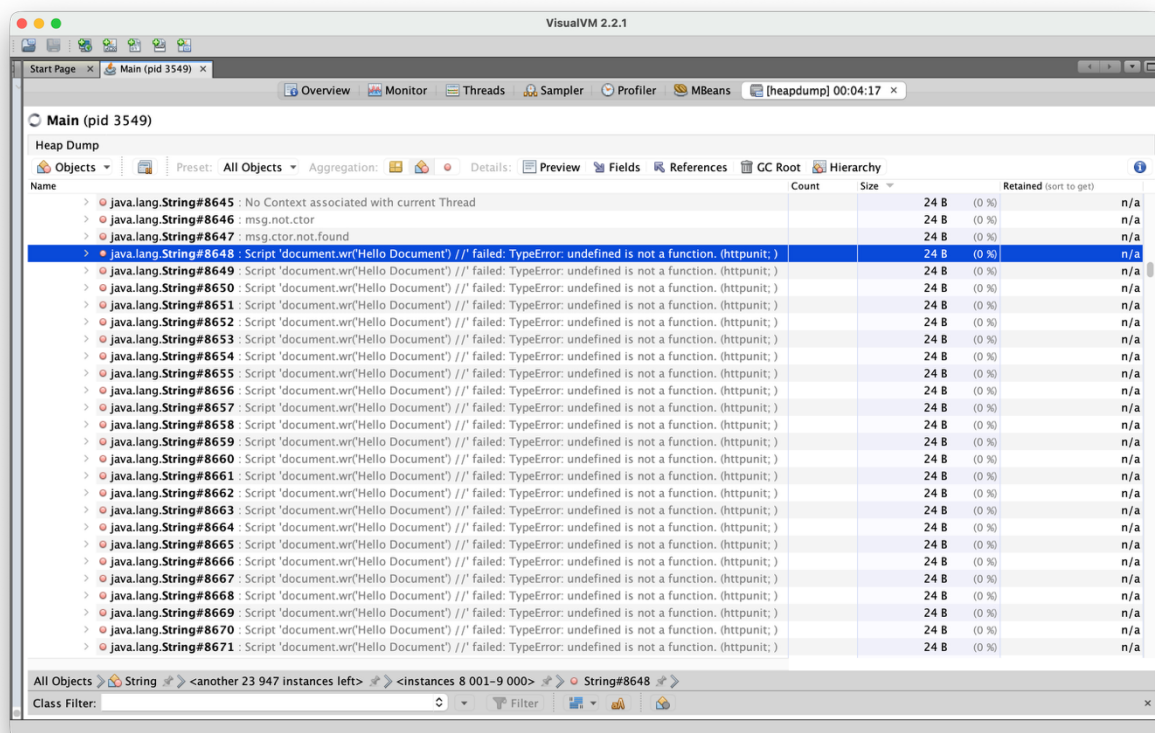


Рис. 15. Обзор `java.lang.String`

При просмотре была обнаружена критическая аномалия: среди строковых объектов присутствуют идентичные строки с сообщением об ошибке `Script 'document.wr('Hello Document')' // failed: TypeError: undefined is not a function. (httpunit; )`. Получается, при каждом HTTP-запросе приложение пытается выполнить JavaScript-сценарий `document.wr('Hello Document')`, однако такого метода не существует (вместо `wr()` должен использоваться `write()`). Каждая неудачная попытка выполнить скрипт порождает новый строковый объект с сообщением об ошибке, которые накапливаются в куче.

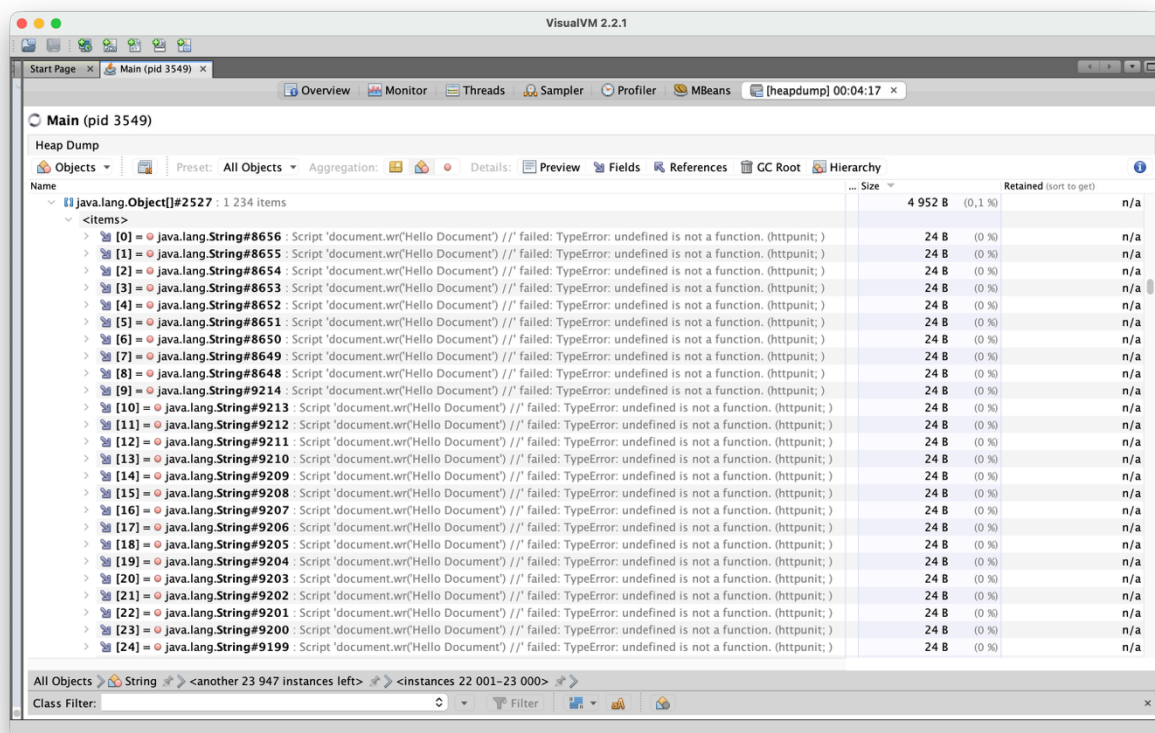


Рис. 16. Обзор `java.lang.Object[]#2527`

При дальнейшем обзоре была обнаружена корневая утечка памяти - `java.lang.Object[]#2527` – при каждом запросе сюда добавляется новая строка с сообщением об ошибке JS, при этом очистка никогда не производится.

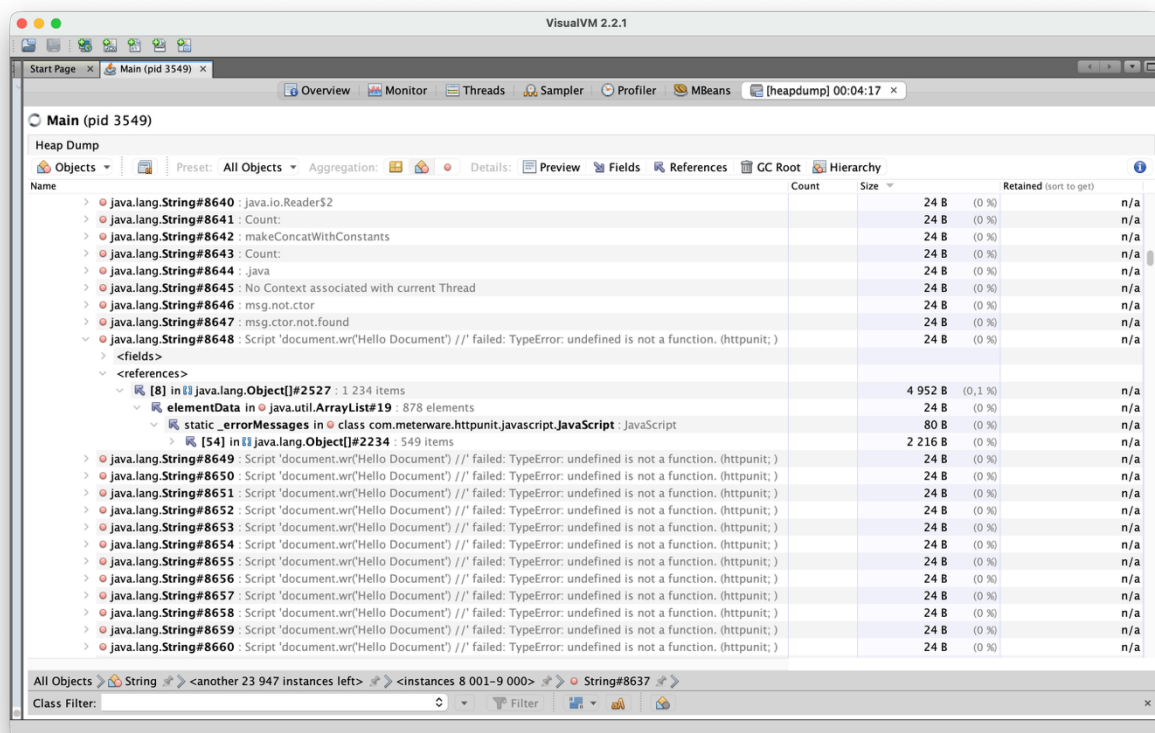


Рис. 17. Обзор <referens> на java.lang.String

Исследование ссылок на строковые объекты (с сообщением об ошибке) позволили получить полную картину происходящего. Статическое поле `_errorMessages` в классе `JavaScript` накапливает сообщения об ошибках JS и никогда не очищается. Каждый неудачный вызов `document.wr()` добавляет новую запись в этот список, который растёт бесконечно до тех пор, пока не происходит `OutOfMemoryError`.

Теперь зайдем в исходный код и исправим ошибки.

Изучение исходного кода класса `com.meterware.httpunit.javascript.JavaScript` подтвердило причину утечки. В классе объявлено статическое поле:

```
private static ArrayList _errorMessages = new ArrayList();
```

Рис. 18. Статическое поле `_errorMessage`

В ходе дальнейшего анализа класса был обнаружен метод `handleScriptException()`, который вызывается каждый раз при возникновении ошибки JS

```

private void handleScriptException( Exception e, String badScript ) { 3 usages
    final String errorMessage = badScript + " failed: " + e;
    if (!(e instanceof EcmaError) && !(e instanceof EvaluatorException)) {
        e.printStackTrace();
        throw new RuntimeException( errorMessage );
    } else if (isThrowExceptionsOnError()) {
        e.printStackTrace();
        throw new ScriptException( errorMessage );
    } else {
        _errorMessages.add( errorMessage );
    }
}

```

Рис. 19. метод handleScriptException

На рисунке 19 можно заметить ветку else. Именно здесь сообщение об ошибке добавляется в список _errorMessage, при этом чистка этого списка нигде не предусмотрена.

Однако в классе JavaScript существует метод clearErrorMessage, предназначенный для очистки накопленных сообщений об ошибках:

```

static void clearErrorMessages() { 1 usage
    _errorMessages.clear();
}

```

Рис. 20. метод clearErrorMessage в классе JavaScript.java

IDE показывает пометку «1 usage». Посмотрим где используется этот метод:

```

public void clearErrorMessages() { 1 usage
    JavaScript.clearErrorMessages();
}

```

Рис. 21. метод clearErrorMessage в классе JavaScriptEngineFactory.java

Он находит в классе JavaScriptEngineFactory. Но здесь опять есть пометка «1 usage». Проследим цепочку дальше:

```

/**
 * Clears the accumulated script error messages.
 */
public static void clearScriptErrorMessages() { no usages
    getScriptingEngine().clearErrorMessages();
}

```

Рис. 22. метод `clearScriptErrorMessage` в классе `HttpUnitOptions.java`

Таким образом, в классе `HttpUnitOption` был обнаружен метод `clearScriptErrorMessage`, который, в свою очередь, имеет пометку «no usage».

Итак, полная цепочка вызовов выглядит следующим образом:

```

clearScriptErrorMessages() → JavaScriptEngineFactory.clearErrorMessages() →
JavaScript.clearErrorMessages() → _errorMessages.clear()

```

Механизм очистки реализован, однако — метод `clearScriptErrorMessages()` — нигде не вызывается. Это означает, что вся цепочка никогда не срабатывает.

Для того чтобы это исправить необходимо добавить вызов `HttpUnitOptions.clearScriptErrorMessages()` в цикл обработки запросов в классе `Main.java`.

```

while (true) {
    WebResponse response = sc.getResponse(request);
    System.out.println("Count: " + number++ + response);
    HttpUnitOptions.clearScriptErrorMessages(); // исправление
    java.lang.Thread.sleep( millis: 200);
}

```

Рис. 23. Исправление в классе `Main.java`

Также исправим ошибку обращение к невалидному JS методу:

```

out.println("document.write('Hello Document')"); // wr → write

```

Рис. 24. Исправление в классе `HelloWorld.java`

Теперь попробуем запустить программу с исправлениями.

При повторном запуске программа снова завершалась с ошибкой, при чем heap начал расти только в тот момент, когда подключалась VisualVM.

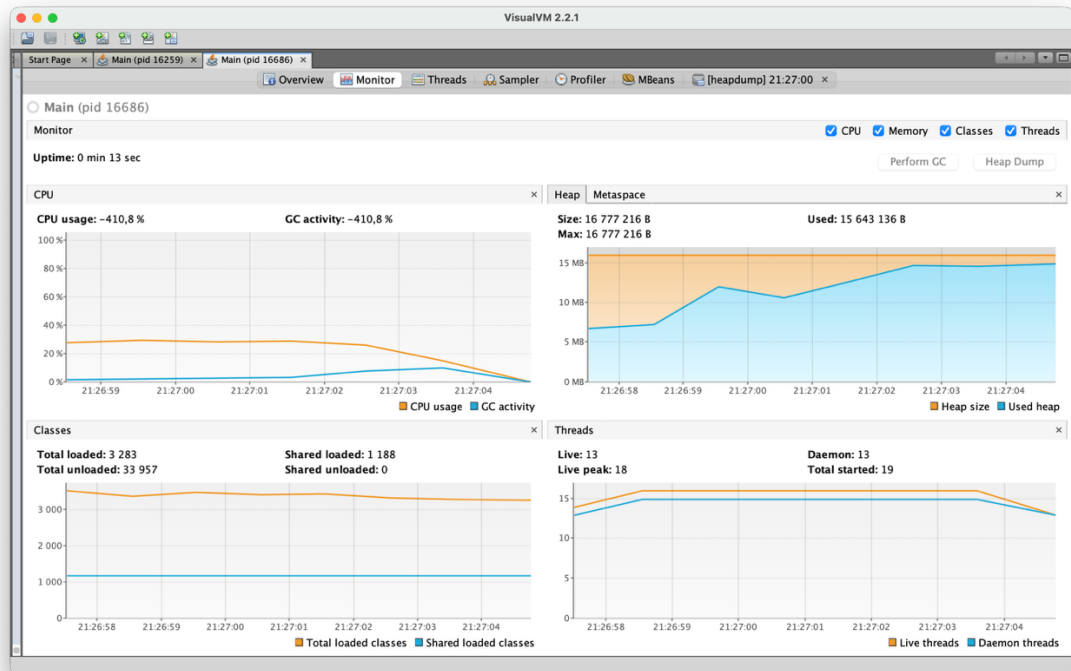


Рис. 25. Мониторинг исправленной программы

На рисунке 25 на графике Heap виден резкий рост. Heap dump показал значительное количество объектов `java.util.TreeMap$Entry`.

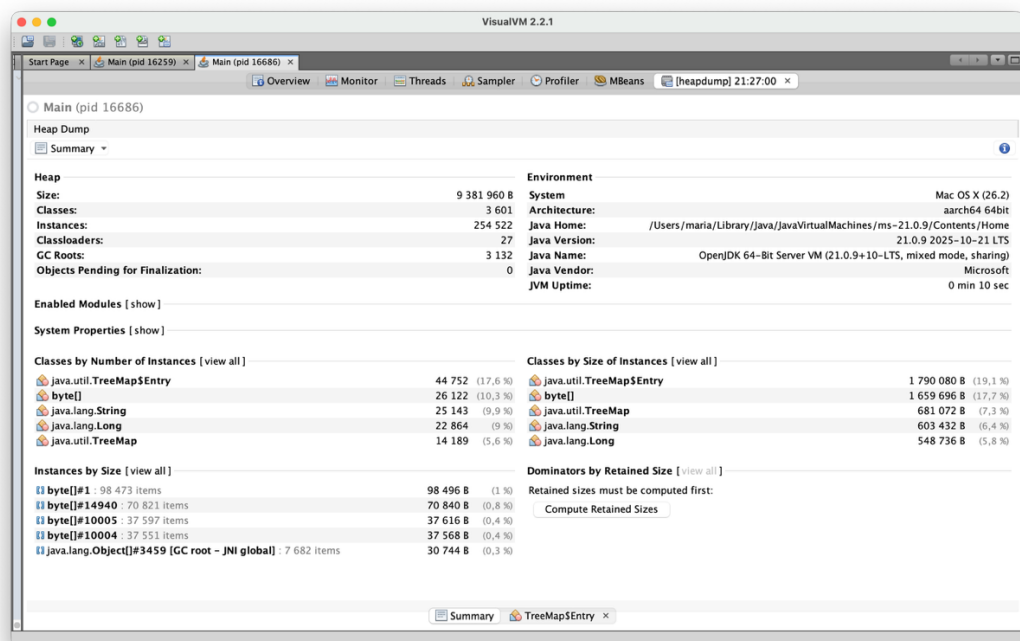


Рис. 26. Heap dump исправленной программы

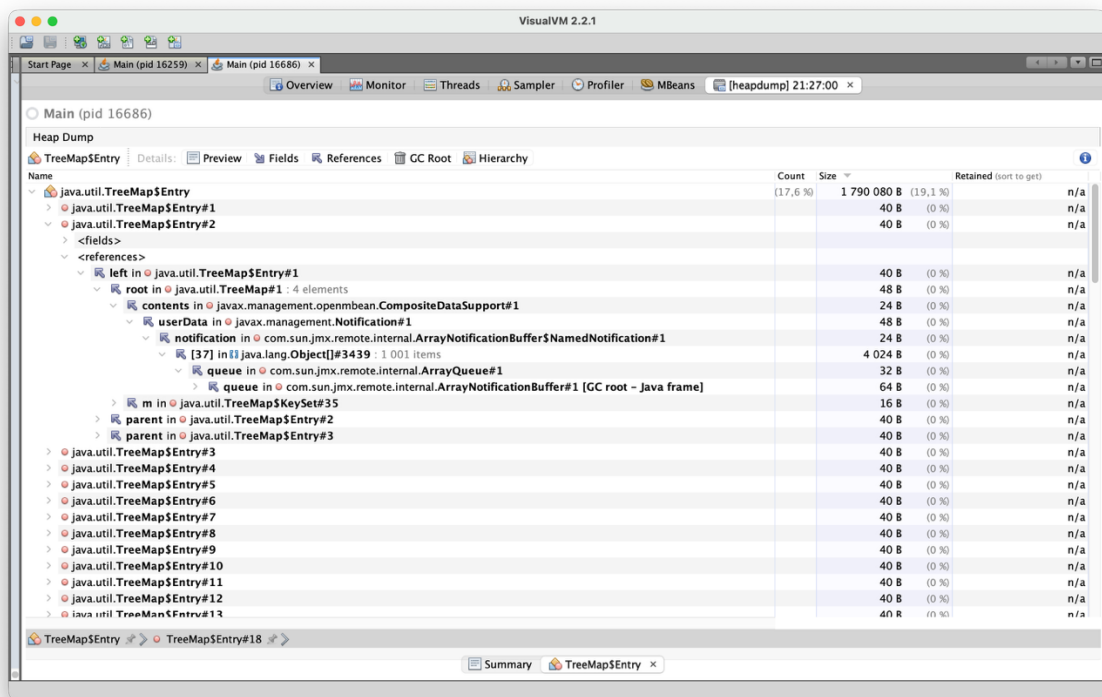


Рис. 27. Анализ объектов `TreeMap$Entry`

Анализ объектов `TreeMap$Entry` показывает, что они связаны с JMX: цепочка ссылок ведёт через `javax.management.Notification` → `ArrayNotificationBuffer` → буфер из 1001 элемента (`Object[]#3439`). Это стандартный буфер уведомлений JMX-сервера, создаваемый при подключении VisualVM (не является проблемой исходного кода программы).

Для решения этой проблемы необходимо запустить программу с параметром - `Djmx.remote.x.notification.buffer.size=10`, который ограничивает размер буфера уведомлений `ArrayNotificationBuffer` до 10 элементов.

После этого количество объектов `TreeMap$Entry` в куче значительно сократилось.

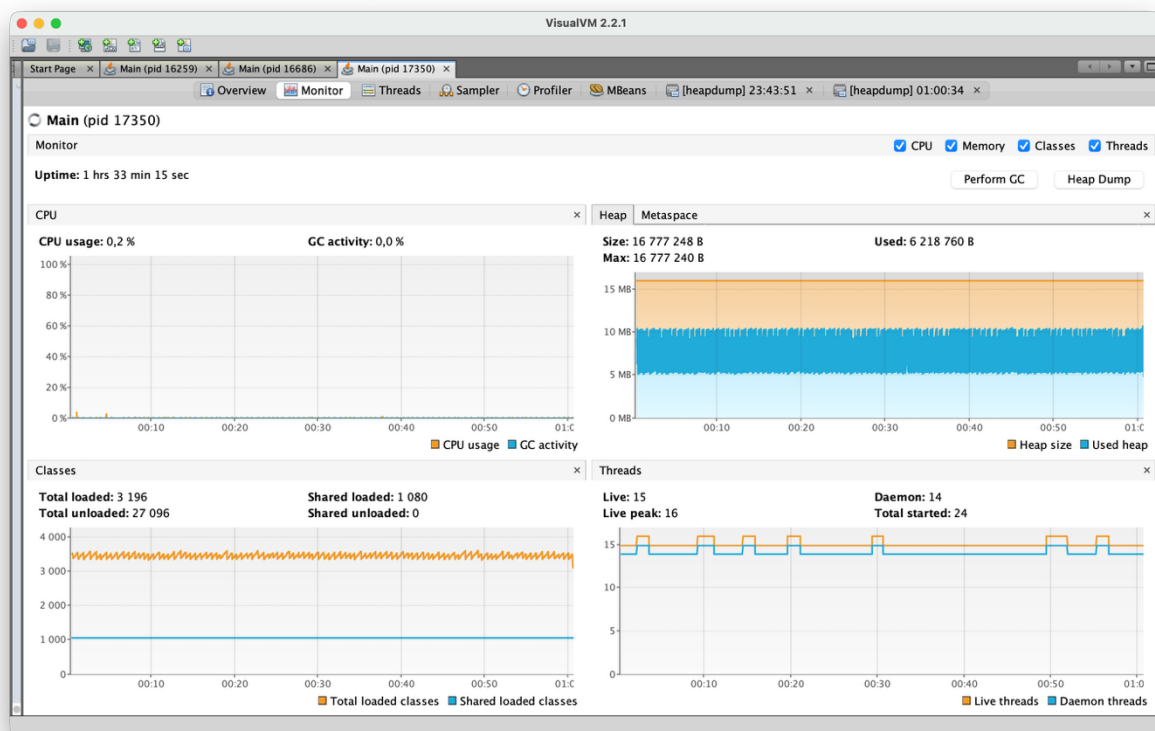


Рис. 28. Мониторинг программы после всех исправлений

Таким образом, можно сделать вывод, что выявленные проблемы с производительностью были успешно локализованы и устранены. График Heap в VisualVM демонстрирует стабильное поведение на протяжении всего времени работы программы: используемая память колеблется в фиксированном диапазоне 5–10 МБ, не превышая выделенный максимум.

- IntelliJ Profiler

Для профилирования программы также был использован встроенный профилировщик – IntelliJ Profiler.

Для наглядности задержка между запросами была убрана.

```
System.out.println("Count: " + number++ + response);  
//      java.lang.Thread.sleep(200);  
//    }  
//  } catch (InterruptedException ex) {  
//    Logger.getLogger("global").log(Level.SEVERE, null, ex);
```

Рис. 29. Закомментированный код Main.java

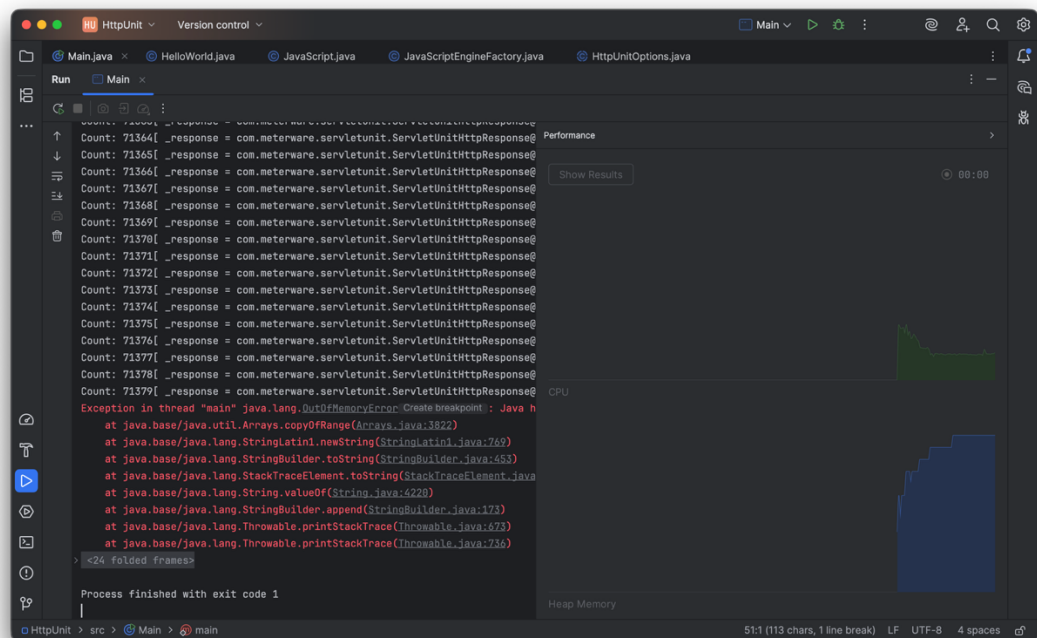


Рис. 30. Крах приложения и график потребления памяти во время работы приложения

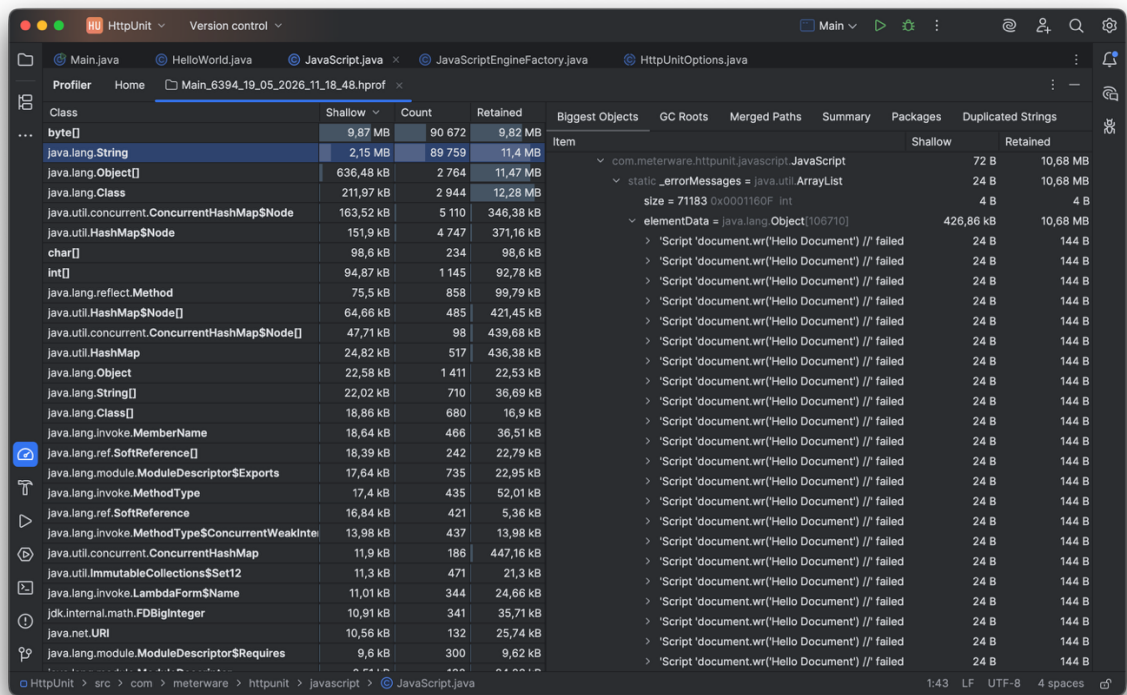


Рис. 31. Анализ heap dump в IntelliJ Profiler

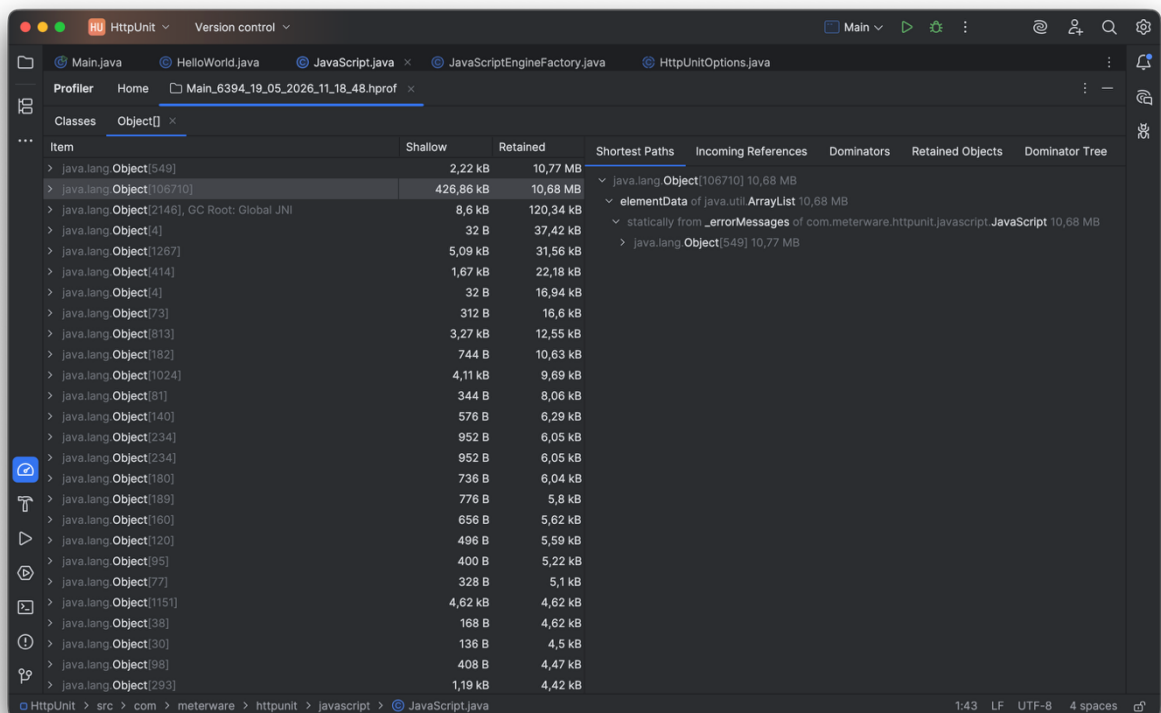


Рис. 32. Обзор объекта java.lang.Object[106710]

В ходе профилирования с помощью встроенного инструмента IntelliJ Profiler была подтверждена ранее выявленная утечка памяти. График потребления памяти демонстрирует непрерывный рост heap вплоть до краша приложения с ошибкой OutOfMemoryError. Анализ снимка кучи показал, что доминирующим объектом по объёму удерживаемой памяти является массив `java.lang.Object[106710]`, связанный с полем `_errorMessages` класса `JavaScript`. Это полностью согласуется с результатами, полученными в VisualVM.

Таким образом, при выполнении задания была проведена диагностика и устранение утечки памяти в приложении `HttpUnit`. С помощью утилит VisualVM и IntelliJ Profiler было установлено, что ошибка OutOfMemoryError следствие бесконтрольного роста поля `_errorMessages` в классе `JavaScript`.

В общем, IntelliJ Profiler является более предпочтительным для профилирования, так как он встроен в среду разработки и не требует отдельного подключения к процессу, что исключает накладные расходы, характерные для VisualVM (подключение VisualVM спровоцировало дополнительный рост heap за счёт буфера уведомлений JMX, что потребовало отдельной настройки параметра). IntelliJ Profiler лишён этого недостатка и позволяет получать достоверные результаты профилирования без вмешательства в поведение исследуемого приложения.

Вывод

Выполнение лабораторной работы по ОПИ позволило более углублённо познакомиться с инструментами мониторинга и профилирования Java-приложений. Данная работа дала возможность применить полученные теоретические знания на практике и развить навыки работы с утилитами JConsole, VisualVM и IntelliJ Profiler. В процессе выполнения я научилась работать с данными утилитами. Приобретённый опыт поможет при дальнейшем изучении ОПИ.